Formation Fine-tuning

Fait par Thibault GAREL

INTELLIGENCE LAB BY ECE

ECE ÉCOLE D'INGÉNIEURS ENGINEERING SCHOOL

Formation théorique

Formation pratique

Projet

Formation théorique

Objectif des NLP

NLP (Natural Language Processing)

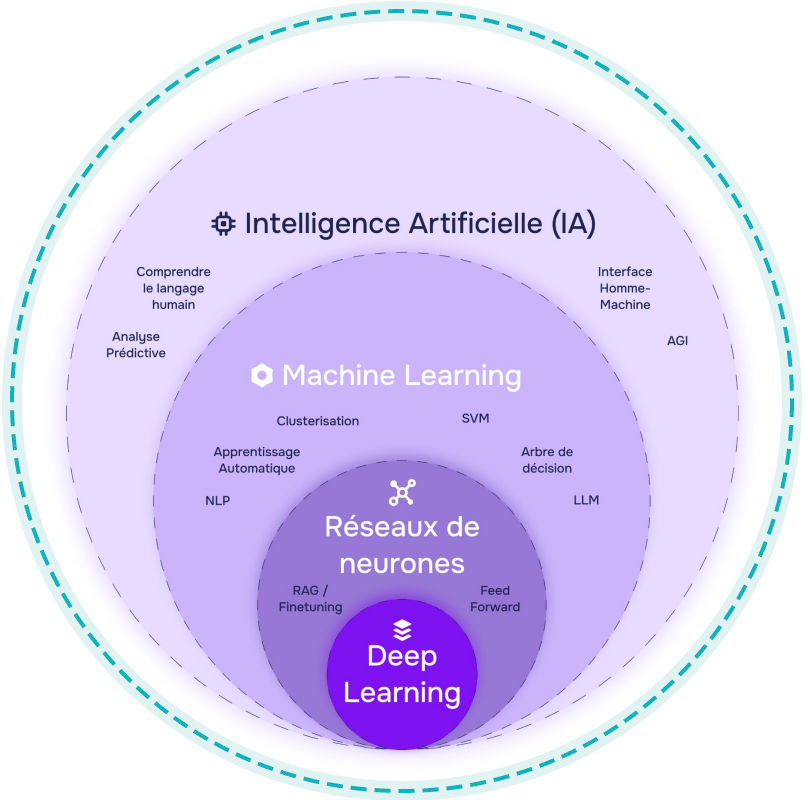
Traitement du Langage Naturel (TLN)

Interaction Homme-Machine

Pont entre l'informatique et le langage humain.

Objectif Principal

Donner aux machines la capacité de **lire**, **comprendre**, **interpréter** et **générer** du langage de manière utile.



Objectif des NLP

Compréhension

Extraire le sens, l'intention ou le sentiment d'un texte.

Génération

Produire du texte de manière autonome (emails, résumés).

Traduction

Convertir un texte d'une langue à une autre de manière fluide.

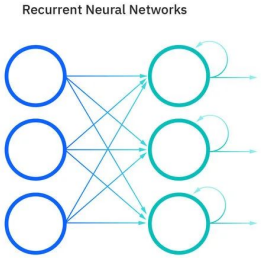
Interaction vocale

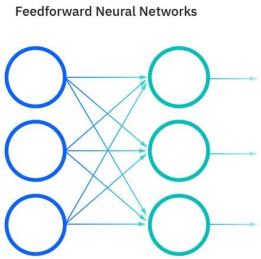
Transformer la voix en texte et inversement (assistants vocaux).

Extraction d'informations

Trouver des données spécifiques dans de grandes masses documentaires.

Architectures précédentes





RNN

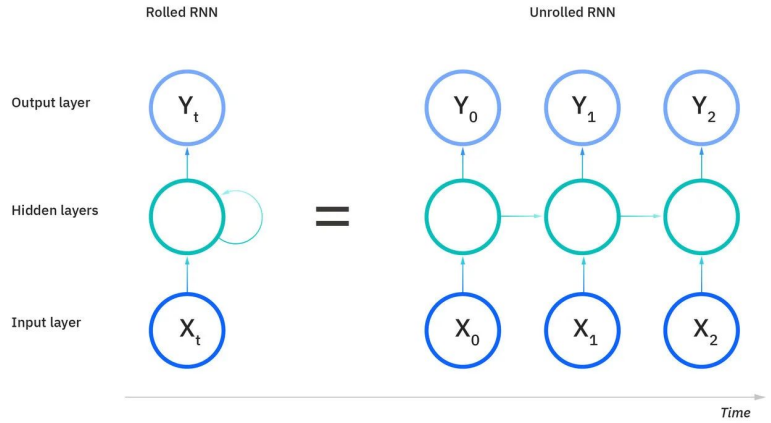
Recurrent Neural Network

Usage principal

Traite les données séquentielles comme le texte (phrases) ou le son (musique).

Caractéristique clé

Possède une **mémoire à court terme** de ce qui vient d'être traité pour comprendre le contexte immédiat.



Architectures précédentes

LSTM

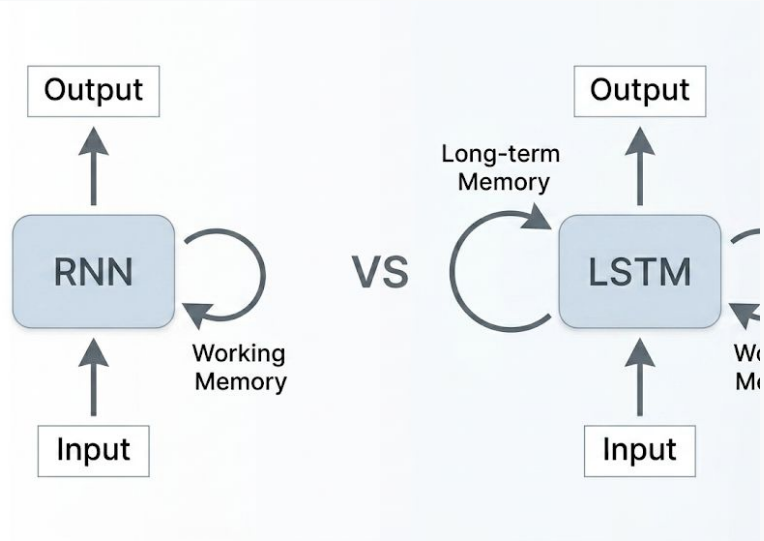
Long Short-Term Memory

Évolution du RNN

Version améliorée du RNN conçue pour résoudre le problème de la perte d'information sur les longues séquences.

3 Décisions par étape :

- **Oublier :** Supprimer l'inutile.
- **Garder :** Stocker l'important.
- **Utiliser :** Mettre à jour la sortie.



Problèmes de ces architectures

1. Le problème du traitement séquentiel : le mur de la lenteur

Les RNN et les LSTM lisent un texte exactement comme vous et moi : **mot par mot, de gauche à droite**. Pour comprendre le mot numéro 10, l'IA est obligée d'avoir fini de traiter les 9 mots précédents.

Pourquoi c'est un problème :

Les ordinateurs modernes (surtout les GPUs) tirent leur puissance de leur capacité à faire des milliers de calculs **en même temps** (en parallèle).

La conséquence :

Il est **impossible de paralléliser le travail**. L'entraînement sur de gros volumes de textes prend un temps infini, créant un goulot d'étranglement matériel.

Problèmes de ces architectures

2. Le problème de la mémoire et du contexte : le goulot d'étranglement

Le LSTM doit compresser tout le contexte passé dans une seule et même "boîte" mathématique de taille fixe : le **vecteur d'état caché**.

Pourquoi c'est un problème :

Imaginez résumer chaque chapitre d'un livre sur un unique **Post-it** pour comprendre le suivant. L'information essentielle finit par être sacrifiée.

La conséquence :

Sur de longs textes, le contexte se dilue. Si le 1000ème mot cite le premier, le LSTM échoue car l'info a été **"écrasée" 999 fois**.

Révolution !

Attention Is All You Need

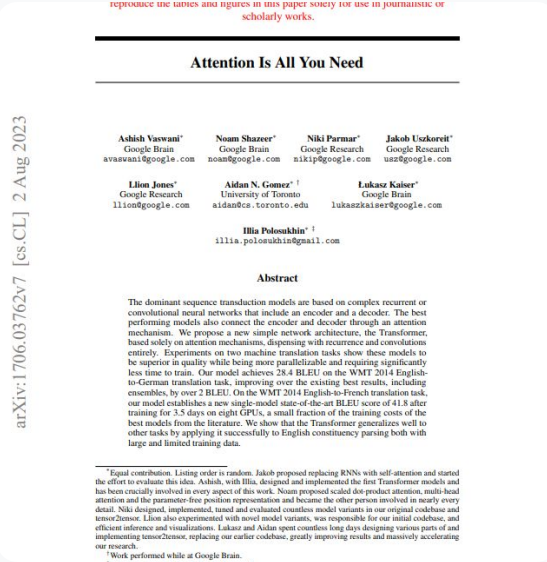
Le papier de recherche séminal qui a introduit l'architecture Transformer, redéfinissant le traitement du langage naturel.

Auteurs clés :

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin.

Consulter l'article original :

arxiv.org/pdf/1706.03762



reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* † illia.polosukhin@gmail.com

arXiv:1706.03762v7 [cs.CL] 2 Aug 2023

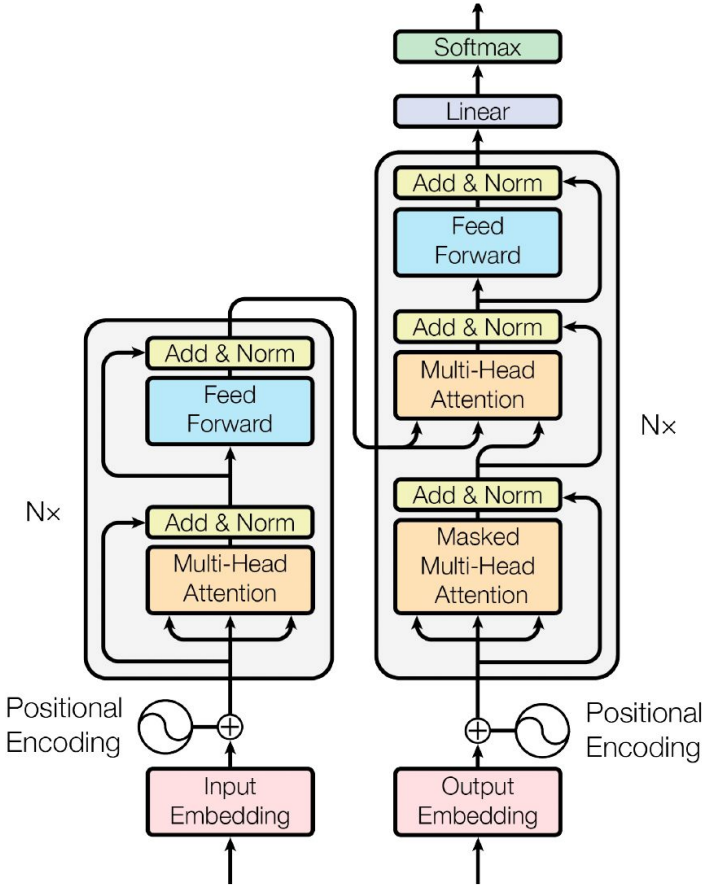
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

†Work performed while at Google Brain.

Formation pratique

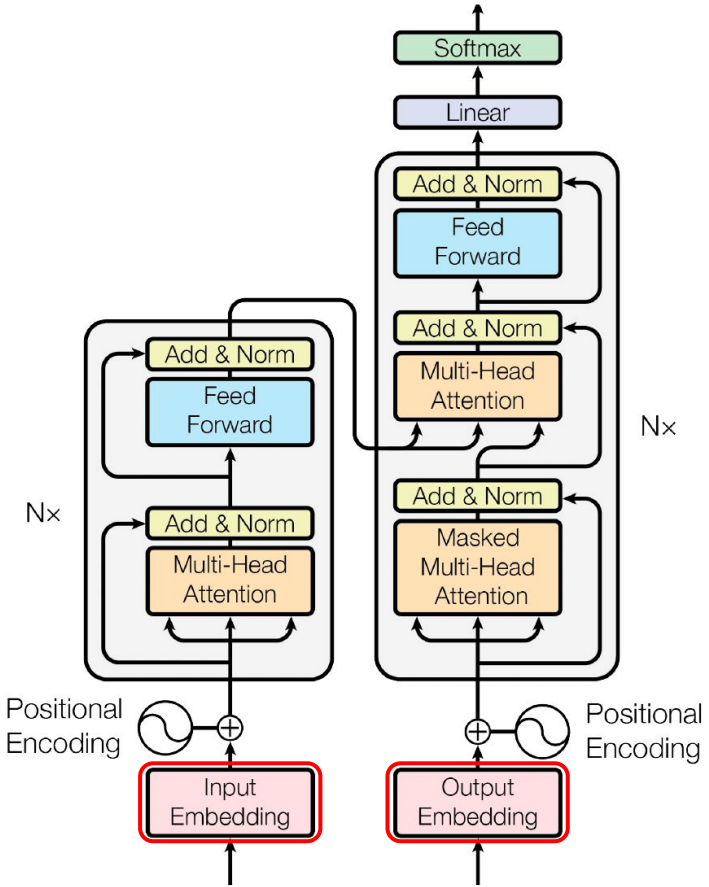


Compréhension de l'Architecture

Point par point

- **Embedding :** Représentation vectorielle des mots.
- **Positional Encoding :** Ajoute l'information sur l'ordre des mots.
- **Multi-Head Attention :** Focus simultané sur différentes parties de la séquence.
- **Feed Forward :** Traitement identique par position via réseaux de neurones.
- **Add & Norm :** Crucial pour la stabilité de l'entraînement.

Encoder / Décoder : Les piliers des architectures modernes (GPT, BERT...).

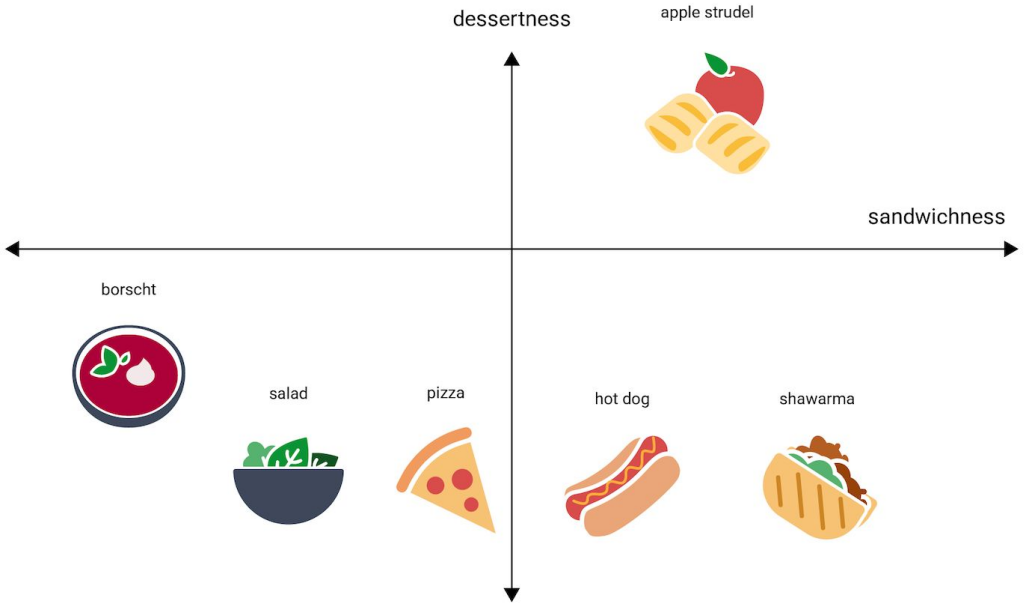


Embedding

L'**embedding** est une méthode permettant de transformer un élément (texte avec des tokens, images, données…) en une **liste de nombres** qui capture sa **signification**, et non sa forme brute.

Cette liste de nombres est un **vecteur multidimensionnel**.

Embedding

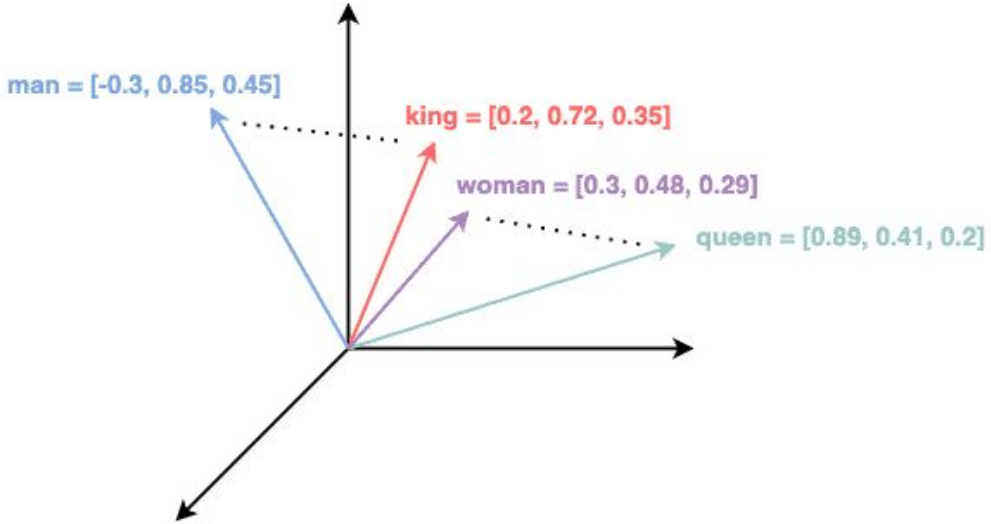


Exemple en **2 dimensions** (taux de ressemblance à un **sandwich** et à un **dessert**)

Attention !
Dans les embeddings utilisés par les IA, il n'y a **pas de sens humain.** Les embeddings sont trouvés par **entraînement** et sont un **mix** de compréhension de **grammaire**, de **syntaxe**, de **sémantique** pour un mot.

D'où la phrase : "We have stopped understanding AI"

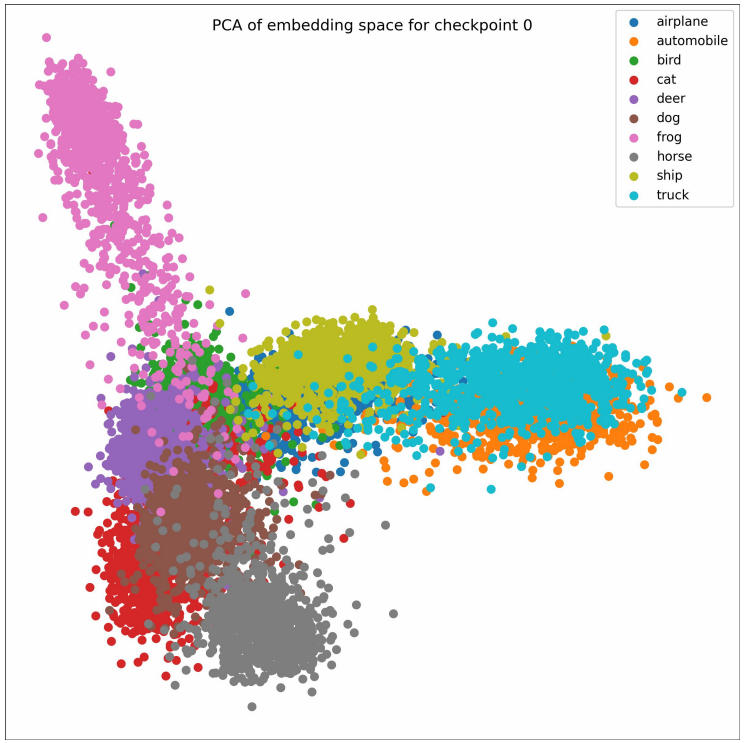
Embedding



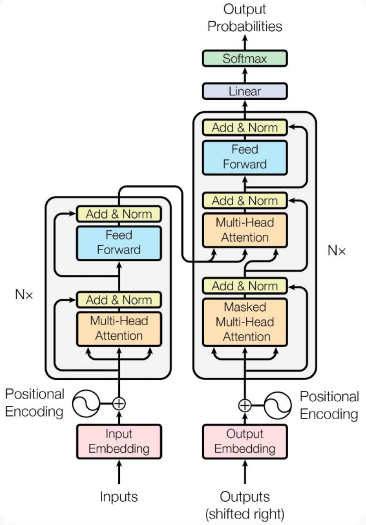
Le but est de pouvoir faire avec un embedding entraîné des **opérations mathématiques** qui vont avoir du **sens dans le langage humain.**

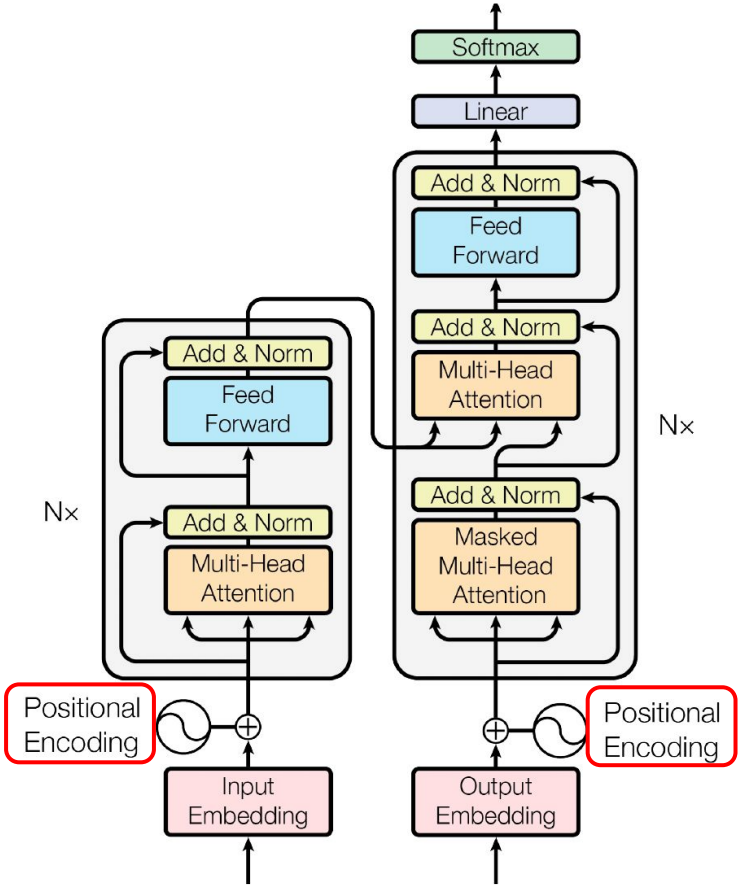
Ex :
E(king) - E(man) + E(woman) ≈ E(queen)

Embedding



Le modèle transformer va modelé cette embedding durant l'entraînement avec la **back propagation de l'erreur global**.





Positional Encoding

Un **transformater** traite une phrase comme un **ensemble de token en parallèle**, et non comme une **séquence**. L'encodeur positionnel sert ici à introduire la **notion d'ordre** dans la représentation des jetons.

"Le chat court dans le jardin" ⇒ Chat avant jardin !

$\text{Input}_i = \text{Embedding}(\text{token}_i) + \text{PositionalEncoding}(i)$

(With i the index of the position)

Positional Encoding

Dans l'article original **"Attention Is All You Need"**, les positions sont encodées avec des sinus et cosinus à différentes fréquences.

Le Calcul :

```
PE(pos, 2i) = sin(pos / 100002i/d)
PE(pos, 2i+1) = cos(pos / 100002i/d)
```

Définitions :

- **pos** : position du jeton (0, 1, 2, ...)
- **i** : index de la dimension
- **d** : dimension totale de l'embedding

Propriétés Utiles :

- Continue et bornée
- Périodique (pas d'explosion numérique)
- Permet d'exprimer un décalage de position comme une transformation linéaire simple

Fréquence (f = 1 / 100002i/d) :

- **Dimension faible** ⇒ f élevée : variations rapides (positions proches distinctes).
- **Dimension élevée** ⇒ f faible : variations lentes (positions proches similaires).

Un même écart peut représenter 1 jeton (f haute) ou un paragraphe entier (f basse).

Positional Encoding

On utilise sinus et cosinus pour définir un angle et une direction. Cela permet une représentation **bijective** de la position sur le cercle unité.

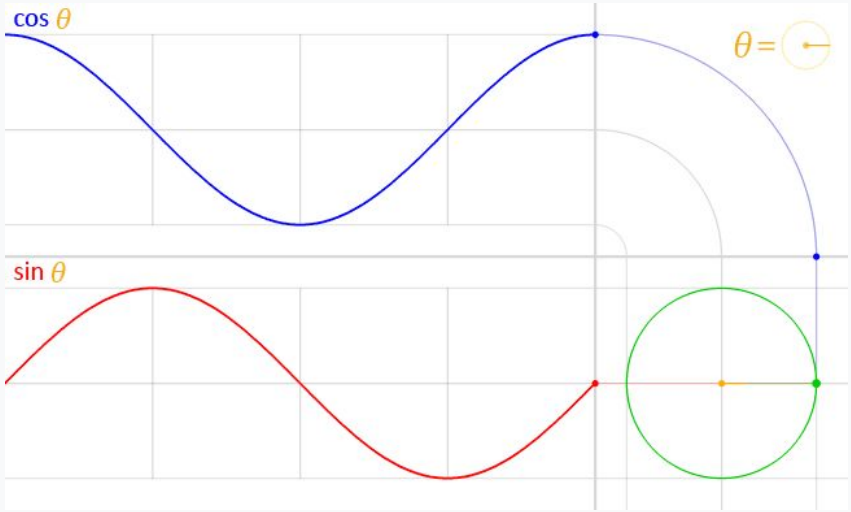
L'encodage de position (PE) est ajouté à l'embedding (E, le sens du jeton) :

`Entrée = E + PE`

Détails mathématiques :

- $E(t) = (e_0, e_1, ..., e_d\text{-}1) \in \mathbb{R}^d$
- $PE(pos) = (p_0, p_1, ..., p_d\text{-}1) \in \mathbb{R}^d$
- $p_0 = \sin(\theta_0)$, $p_1 = \cos(\theta_0)$...
  (sin pour tous les pairs et cos impairs)

Bien que mélangée à l'embedding, le Transformer apprend à extraire cette information !



Positional Encoding

On utilise sinus et cosinus pour définir un angle et une direction. Cela permet une représentation **bijective** de la position sur le cercle unité.

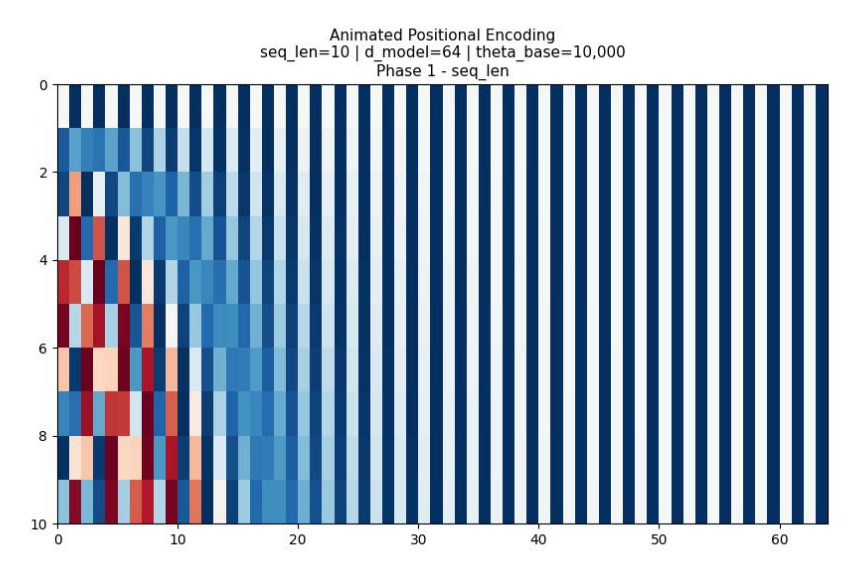
L'encodage de position (PE) est ajouté à l'embedding (E, le sens du jeton) :

`Entrée = E + PE`

Détails mathématiques :

- $E(t) = (e_0, e_1, ..., e_d\text{-}1) \in \mathbb{R}^d$
- $PE(pos) = (p_0, p_1, ..., p_d\text{-}1) \in \mathbb{R}^d$
- $p_o = \sin(\theta_o)$, $p_1 = \cos(\theta_o)$...
  (sin pour tous les pairs et cos impairs)

Bien que mélangée à l'embedding, le Transformer apprend à extraire cette information !



Positional Encoding

Évolution du Positional Encoding

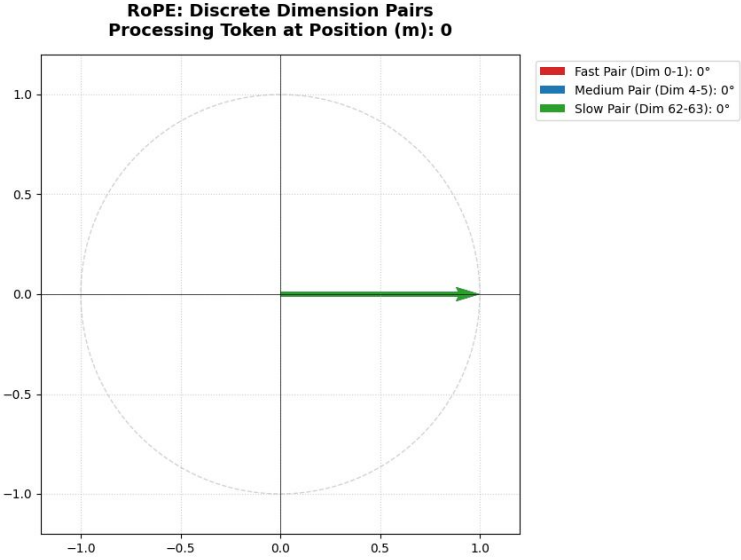
Le Positional Encoding (PE) est crucial pour que le Transformer comprenne l'ordre des mots, car il traite les données en parallèle.

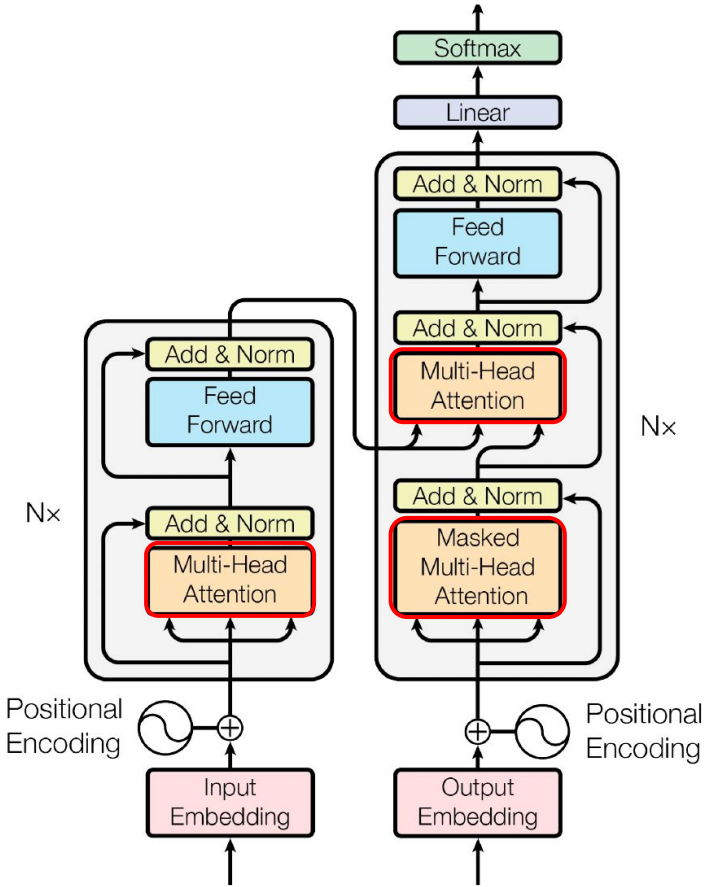
L'approche classique utilise des fonctions sinusoïdales pour encoder la position absolue :

`Entrée = Embedding + PE`

Nouveaux Encodages (RoPE, ALiBi) :

- **Rotary Positional Embedding (RoPE)** : Applique une rotation aux vecteurs Query et Key.
- **ALiBi** : Ajoute un biais linéaire aux scores d'attention basé sur la distance.
- Permettent une meilleure **extrapolation** sur des séquences plus longues.





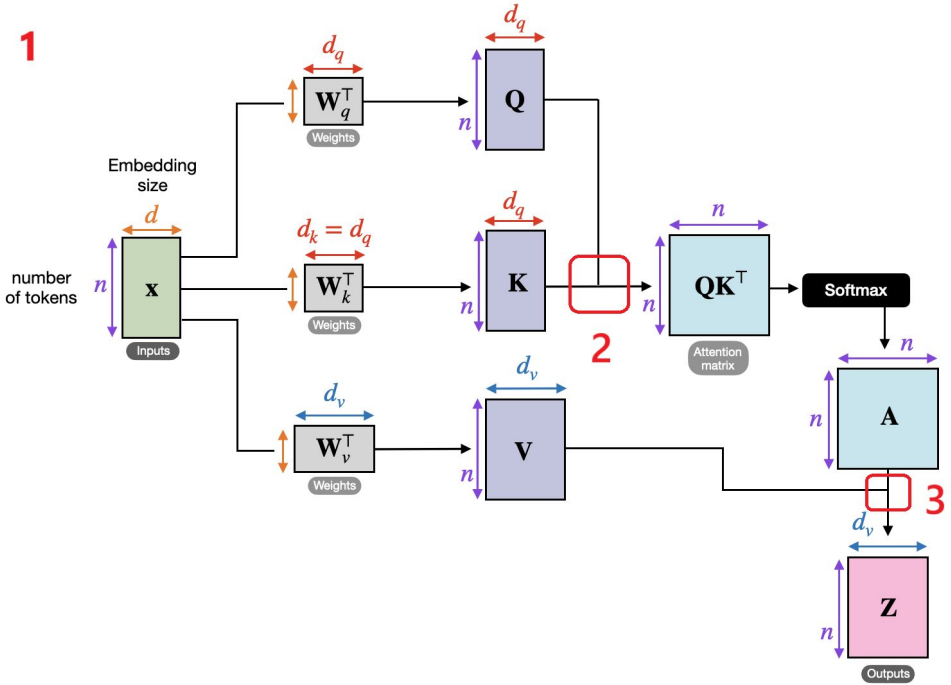
Head Attention

Dans un Transformer, le mécanisme central est **l'attention**. Ce mécanisme repose sur trois vecteurs dérivés de l'entrée :

- Q (Query),
- K (Key),
- V (Value).

Ils déterminent comment chaque token (mot, sous-mot, etc.) interagit avec les autres éléments de la même séquence. Comprend les liens entre les mots.

Head Attention

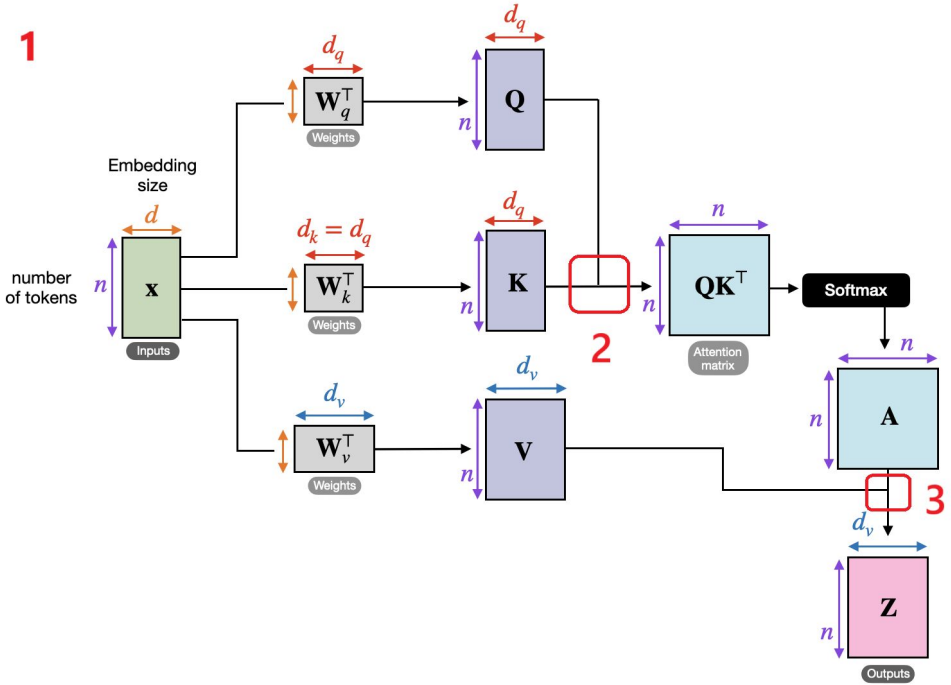


Q = Query → Que recherche-t-on ?
La question qu'un token pose pour trouver le contexte pertinent.

K = Key → Que contient-il ?
Un label qui représente le type d'information que contient le token.

V = Value → Que propose-je ?
Le contenu informationnel concret qui peut être partagé si on y prête attention.

Head Attention

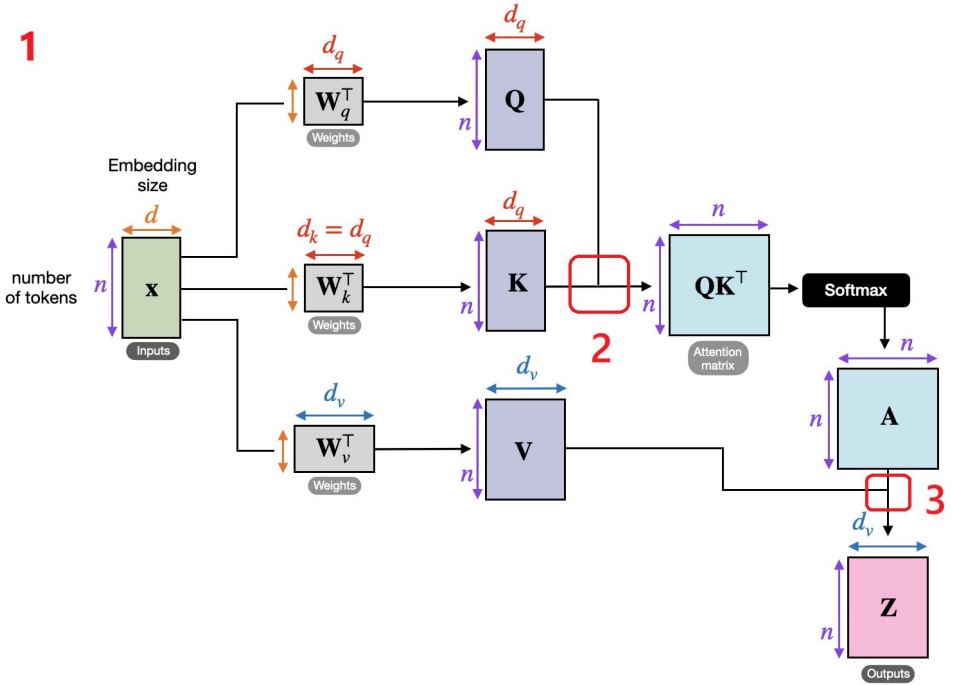


Q pose une question : « Qui, dans la séquence, peut m'aider ?»
(Ex : « Y a-t-il des adjectifs autour de moi ? »)

K fournit une identité : « Je peux vous aider si vous avez besoin de contexte concernant X. »
(Ex : « Oui, je suis un adjective ! »)

V fournit un contenu : « Voici ce que je peux apporter. »
(Ex : Je peux dire qu'un élément de la phrase est bleu.)

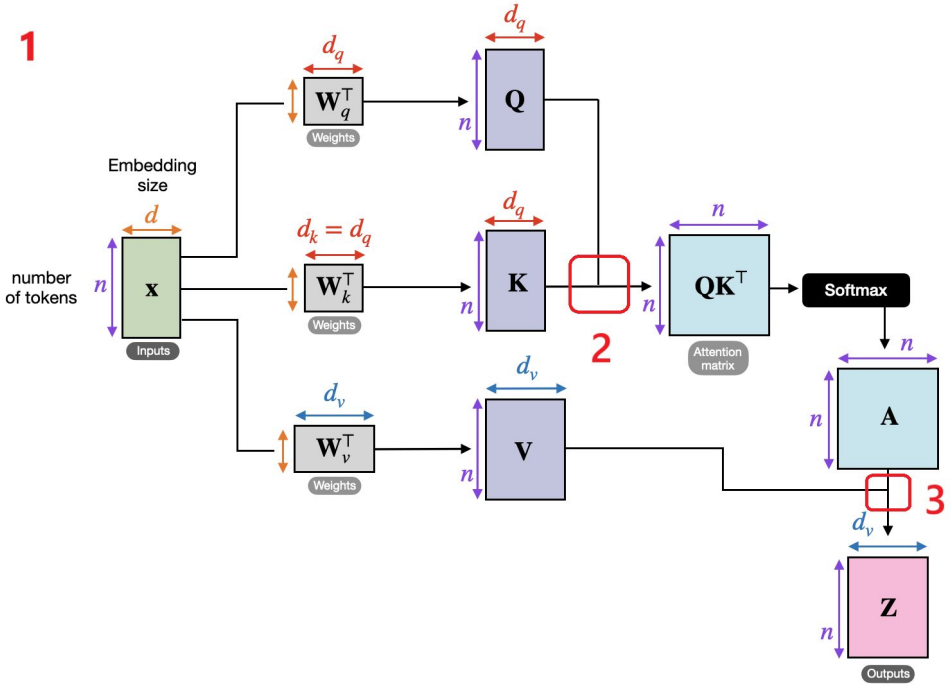
Head Attention

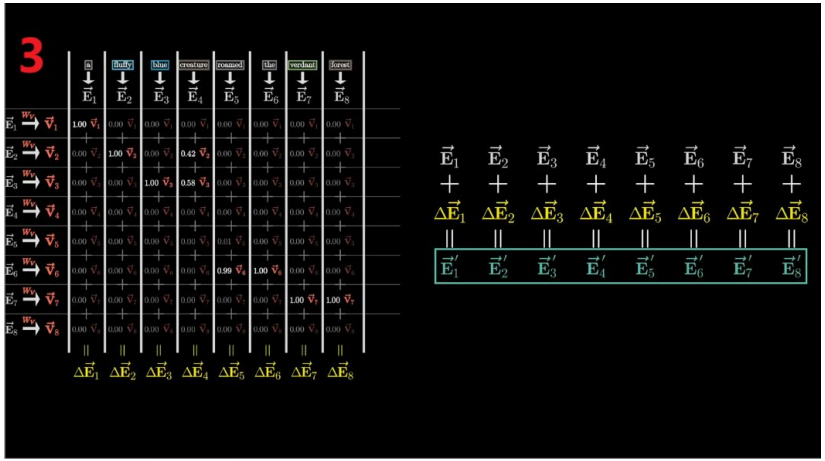


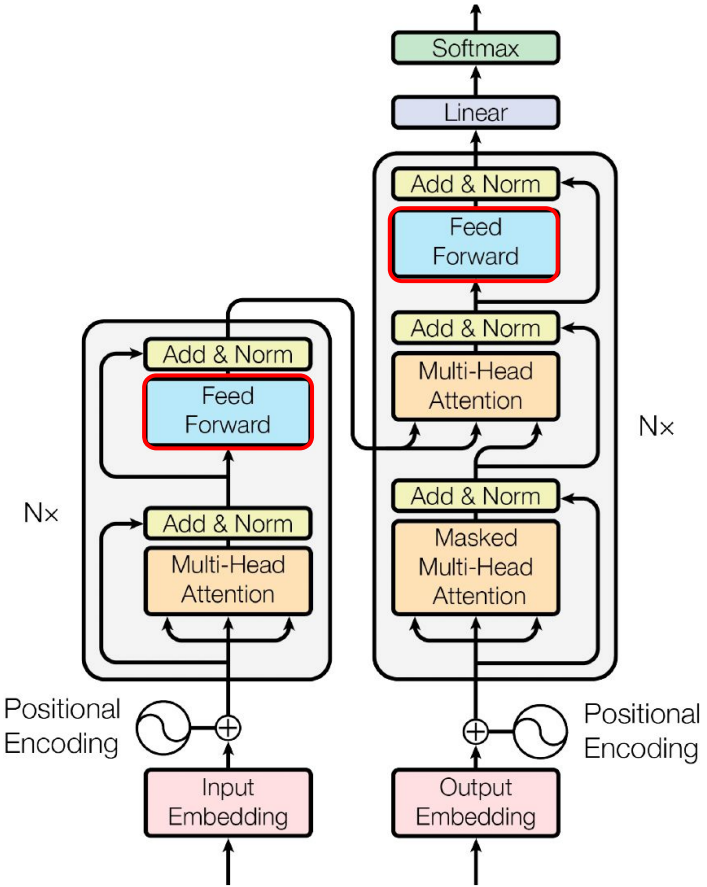
2

	a	fluffy	blue	creature	roamed	the	verdant	forest
	$\vec{E}_1 \downarrow W_Q \vec{Q}_1$	$\vec{E}_2 \downarrow W_Q \vec{Q}_2$	$\vec{E}_3 \downarrow W_Q \vec{Q}_3$	$\vec{E}_4 \downarrow W_Q \vec{Q}_4$	$\vec{E}_5 \downarrow W_Q \vec{Q}_5$	$\vec{E}_6 \downarrow W_Q \vec{Q}_6$	$\vec{E}_7 \downarrow W_Q \vec{Q}_7$	$\vec{E}_8 \downarrow W_Q \vec{Q}_8$
a $\rightarrow \vec{E}_1 \xrightarrow{W_k} \vec{K}_1$	$\vec{K}_1 \cdot \vec{Q}_1$	$\vec{K}_1 \cdot \vec{Q}_2$	$\vec{K}_1 \cdot \vec{Q}_3$	$\vec{K}_1 \cdot \vec{Q}_4$	$\vec{K}_1 \cdot \vec{Q}_5$	$\vec{K}_1 \cdot \vec{Q}_6$	$\vec{K}_1 \cdot \vec{Q}_7$	$\vec{K}_1 \cdot \vec{Q}_8$
fluffy $\rightarrow \vec{E}_2 \xrightarrow{W_k} \vec{K}_2$	$\vec{K}_2 \cdot \vec{Q}_1$	$\vec{K}_2 \cdot \vec{Q}_2$	$\vec{K}_2 \cdot \vec{Q}_3$	$\vec{K}_2 \cdot \vec{Q}_4$	$\vec{K}_2 \cdot \vec{Q}_5$	$\vec{K}_2 \cdot \vec{Q}_6$	$\vec{K}_2 \cdot \vec{Q}_7$	$\vec{K}_2 \cdot \vec{Q}_8$
blue $\rightarrow \vec{E}_3 \xrightarrow{W_k} \vec{K}_3$	$\vec{K}_3 \cdot \vec{Q}_1$	$\vec{K}_3 \cdot \vec{Q}_2$	$\vec{K}_3 \cdot \vec{Q}_3$	$\vec{K}_3 \cdot \vec{Q}_4$	$\vec{K}_3 \cdot \vec{Q}_5$	$\vec{K}_3 \cdot \vec{Q}_6$	$\vec{K}_3 \cdot \vec{Q}_7$	$\vec{K}_3 \cdot \vec{Q}_8$
creature $\rightarrow \vec{E}_4 \xrightarrow{W_k} \vec{K}_4$	$\vec{K}_4 \cdot \vec{Q}_1$	$\vec{K}_4 \cdot \vec{Q}_2$	$\vec{K}_4 \cdot \vec{Q}_3$	$\vec{K}_4 \cdot \vec{Q}_4$	$\vec{K}_4 \cdot \vec{Q}_5$	$\vec{K}_4 \cdot \vec{Q}_6$	$\vec{K}_4 \cdot \vec{Q}_7$	$\vec{K}_4 \cdot \vec{Q}_8$
roamed $\rightarrow \vec{E}_5 \xrightarrow{W_k} \vec{K}_5$	$\vec{K}_5 \cdot \vec{Q}_1$	$\vec{K}_5 \cdot \vec{Q}_2$	$\vec{K}_5 \cdot \vec{Q}_3$	$\vec{K}_5 \cdot \vec{Q}_4$	$\vec{K}_5 \cdot \vec{Q}_5$	$\vec{K}_5 \cdot \vec{Q}_6$	$\vec{K}_5 \cdot \vec{Q}_7$	$\vec{K}_5 \cdot \vec{Q}_8$
the $\rightarrow \vec{E}_6 \xrightarrow{W_k} \vec{K}_6$	$\vec{K}_6 \cdot \vec{Q}_1$	$\vec{K}_6 \cdot \vec{Q}_2$	$\vec{K}_6 \cdot \vec{Q}_3$	$\vec{K}_6 \cdot \vec{Q}_4$	$\vec{K}_6 \cdot \vec{Q}_5$	$\vec{K}_6 \cdot \vec{Q}_6$	$\vec{K}_6 \cdot \vec{Q}_7$	$\vec{K}_6 \cdot \vec{Q}_8$
verdant $\rightarrow \vec{E}_7 \xrightarrow{W_k} \vec{K}_7$	$\vec{K}_7 \cdot \vec{Q}_1$	$\vec{K}_7 \cdot \vec{Q}_2$	$\vec{K}_7 \cdot \vec{Q}_3$	$\vec{K}_7 \cdot \vec{Q}_4$	$\vec{K}_7 \cdot \vec{Q}_5$	$\vec{K}_7 \cdot \vec{Q}_6$	$\vec{K}_7 \cdot \vec{Q}_7$	$\vec{K}_7 \cdot \vec{Q}_8$
forest $\rightarrow \vec{E}_8 \xrightarrow{W_k} \vec{K}_8$	$\vec{K}_8 \cdot \vec{Q}_1$	$\vec{K}_8 \cdot \vec{Q}_2$	$\vec{K}_8 \cdot \vec{Q}_3$	$\vec{K}_8 \cdot \vec{Q}_4$	$\vec{K}_8 \cdot \vec{Q}_5$	$\vec{K}_8 \cdot \vec{Q}_6$	$\vec{K}_8 \cdot \vec{Q}_7$	$\vec{K}_8 \cdot \vec{Q}_8$

Head Attention







Feed Forward

La Self-Attention

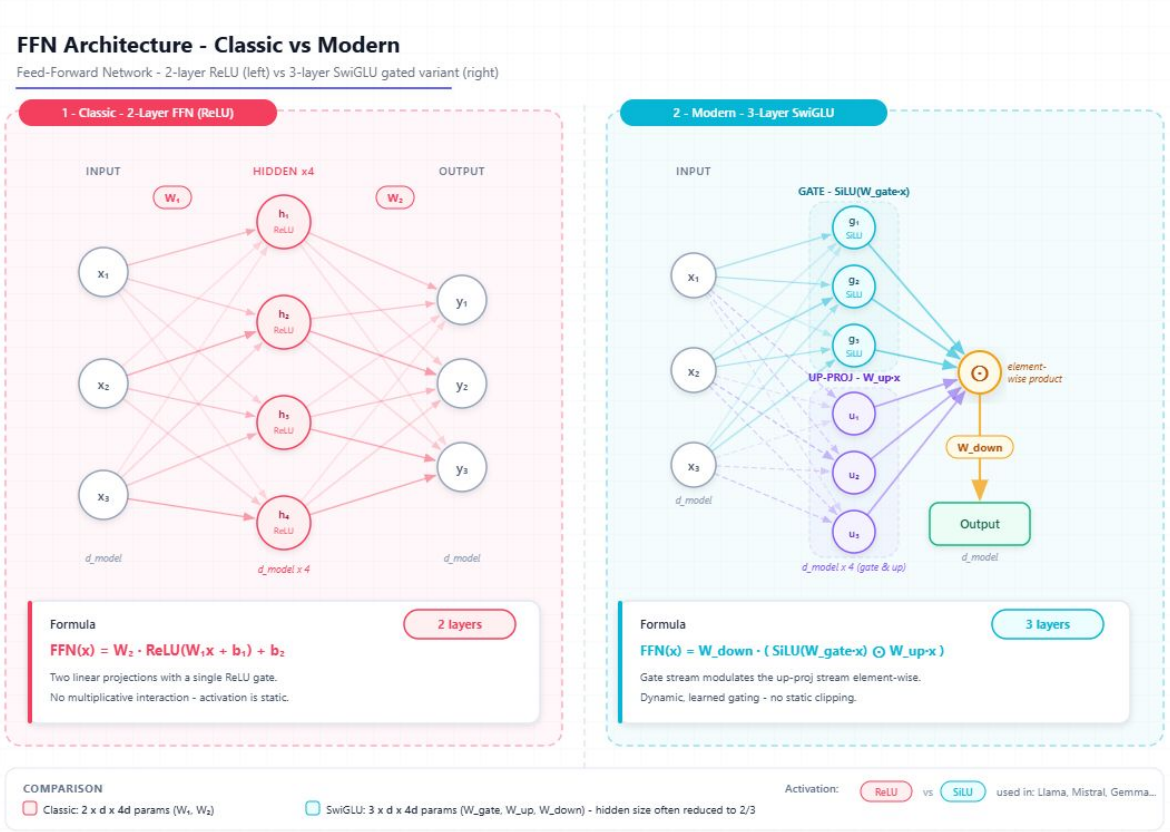
Fait communiquer les mots entre eux pour comprendre le **contexte** (détermine qui fait quoi dans la phrase).

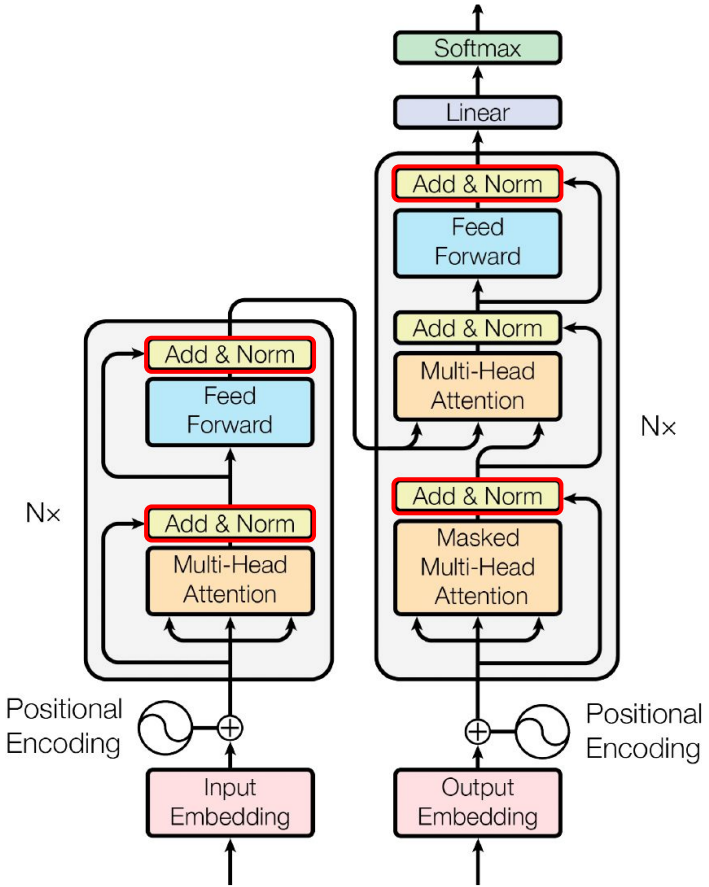
Le Feed-Forward

Traite l'information de manière individuelle et stocke les **connaissances générales** du modèle.

Sert de "mémoire encyclopédique" / calcul au réseau de neurones.

Feed Forward





Add & Norm

Add (Somme Résiduelle)

On ajoute l'embedding calculé après la multi-head attention à l'entrée initiale pour ne pas "oublier" le prompt d'origine.

Norm (Layer Normalization)

On normalise la couche pour centrer et redimensionner les valeurs.

Cela stabilise et accélère la convergence du réseau.

Add & Norm

La normalisation utilisée dans chaque Transformer est la **LayerNorm**, et non la BatchNorm. L'opération reste identique : centrer les données (moyenne = 0) et les redimensionner (écart-type = 1), comme une distribution normale standard N(0, 1) 📊.

La seule différence réside dans l'axe sur lequel on calcule la moyenne :

BatchNorm (axis=0)

Moyenne sur chaque dimension (le batch). Dépend de la taille du batch et des autres tokens.

LayerNorm (axis=1)

Moyenne sur le point lui-même. Indépendant du batch (fonctionne en inférence avec batch_size=1).

$$\mu = \text{mean}(x_1, \ldots, x_d)$$
$$\sigma^2 = \text{var}(x_1, \ldots, x_d)$$
$$y_i = \gamma_i \times (x_i - \mu) / \text{sqrt}(\sigma^2 + \varepsilon) + \beta_i$$

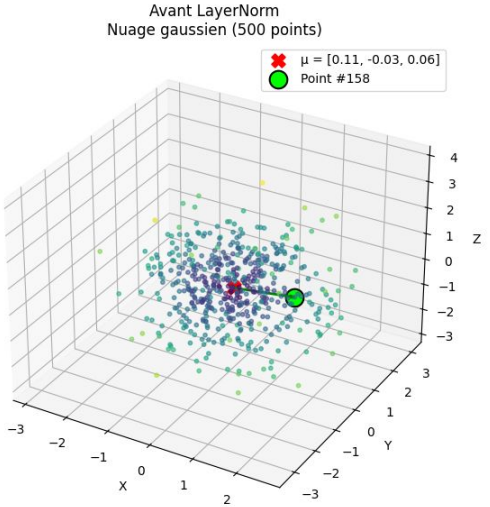
γ (gain) et **β (biais)** : paramètres entraînables permettant d'ajuster l'étalement ou le décalage.

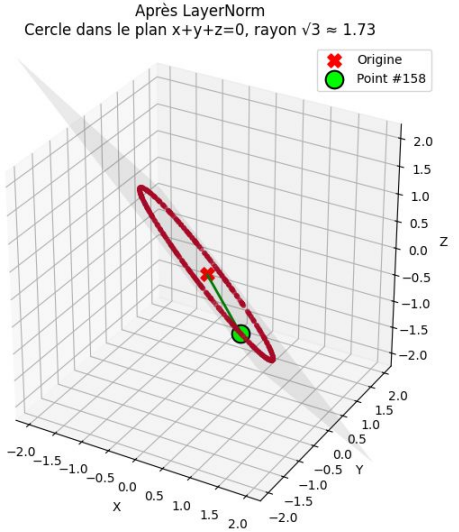
Add & Norm

Est-ce que LayerNorm détruit l'identité du token ?

Le point vert montre le même token avant/après LayerNorm : sa position change radicalement, mais sa direction est préservée.

- **Le sens est dans la direction :** Le contenu sémantique est encodé par l'angle, pas la norme. L'attention ($Q \cdot K^T$) mesure la similarité cosinus, elle ne "voit" que les directions.
- **Perte d'information minime :** LayerNorm retire 2 degrés de liberté. En 3D, c'est brutal (3→1). Mais en 512D (BERT), 99.6% de l'information directionnelle est conservée.
- **Elasticité apprise :** Les paramètres γ et β permettent au réseau de ré-étirer et décaler chaque dimension après normalisation selon ses besoins.





Projet

Projet facile

LLM from Scratch — Modèles de langage construits depuis zéro 🤖

1. Bigram (Simple)

Table de recherche rapide, sans contexte.

- Dataset: Harry Potter
- 10k itér., AdamW

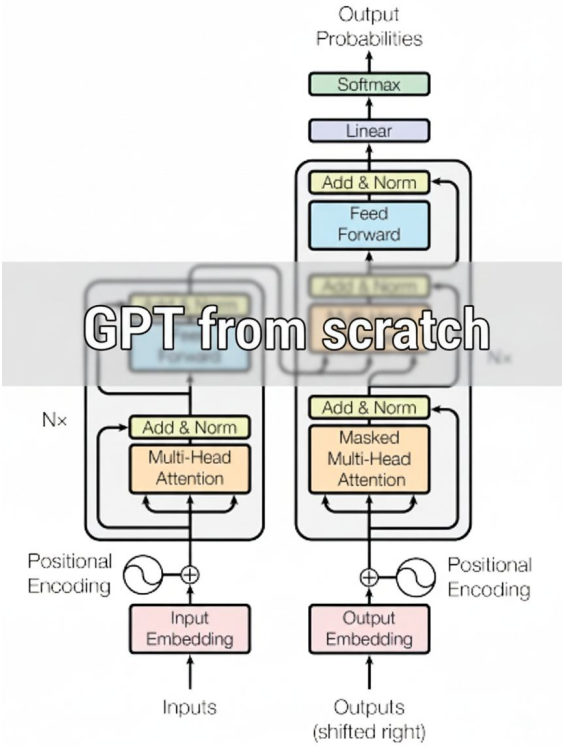
2. Transformer

Architecture GPT-like avec contexte.

- 6 têtes, 6 couches
- Dataset: Victor Hugo
- Support GPU/CUDA

L'Objectif

Montrer l'évolution d'un modèle de base vers un **Transformer** capable de générer du texte cohérent en français 🇫🇷



Projet facile

Accéder au code source

Retrouvez l'intégralité du projet, des codes Transformer et les explications sur GitHub.

Voir le dépôt GitHub

N’hésitez pas à mettre des Stars ⭐!



Merci pour votre écoute

INTELLIGENCE
LAB
BY ECE

ECE
ÉCOLE D'INGÉNIEURS
ENGINEERING SCHOOL